

学以致用



JavaFX集成JDBC开发数据库应用程序

北京理工大学计算机学院
金旭亮

概述

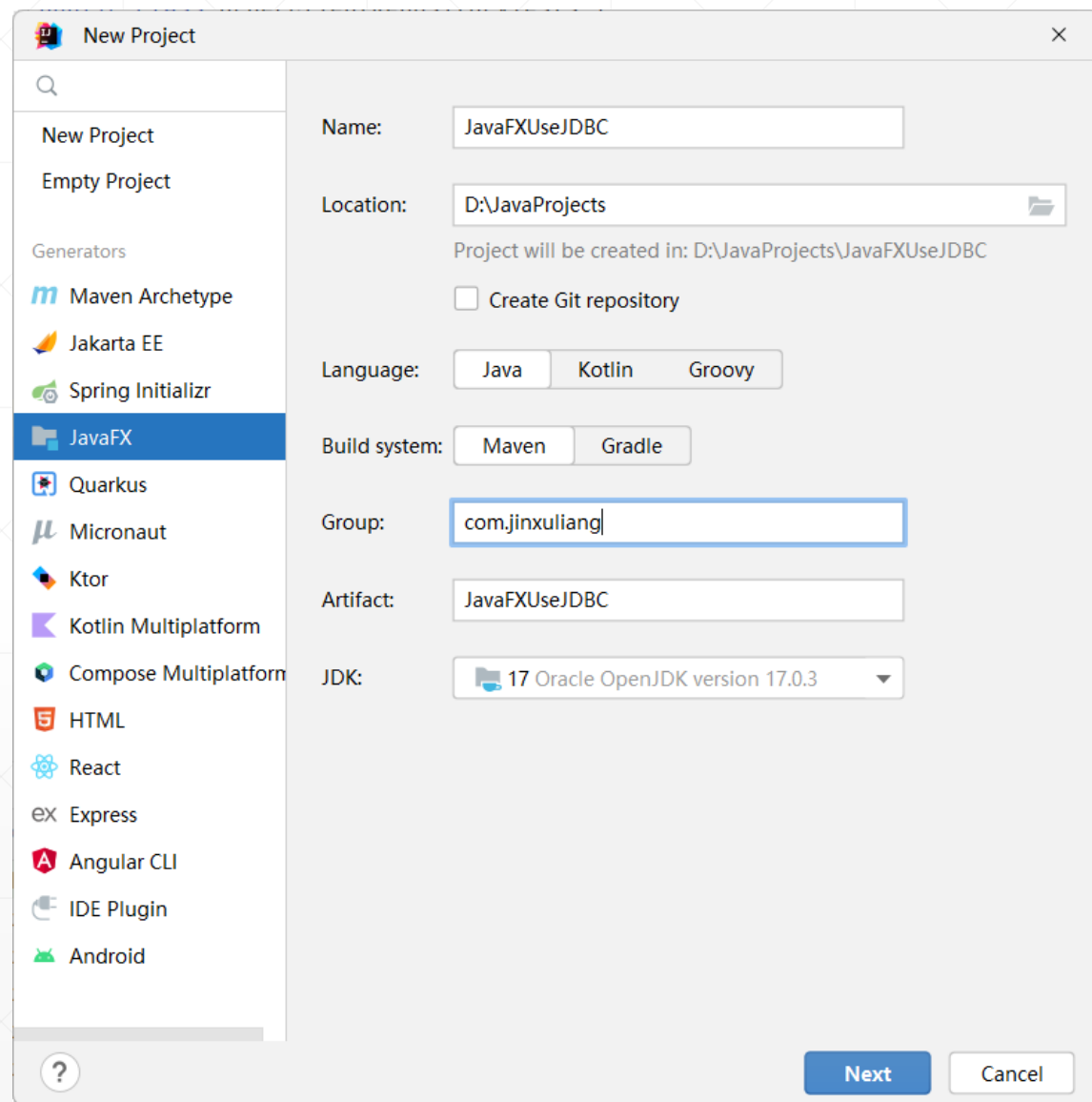


给JavaFX集成JDBC，就能轻松地访问关系型数据库。



本讲介绍一个JavaFX示例，集成了JDBC，访问一个SQLite数据库，并且实现了基础的CRUD功能。

创建JavaFX项目



添加项目依赖



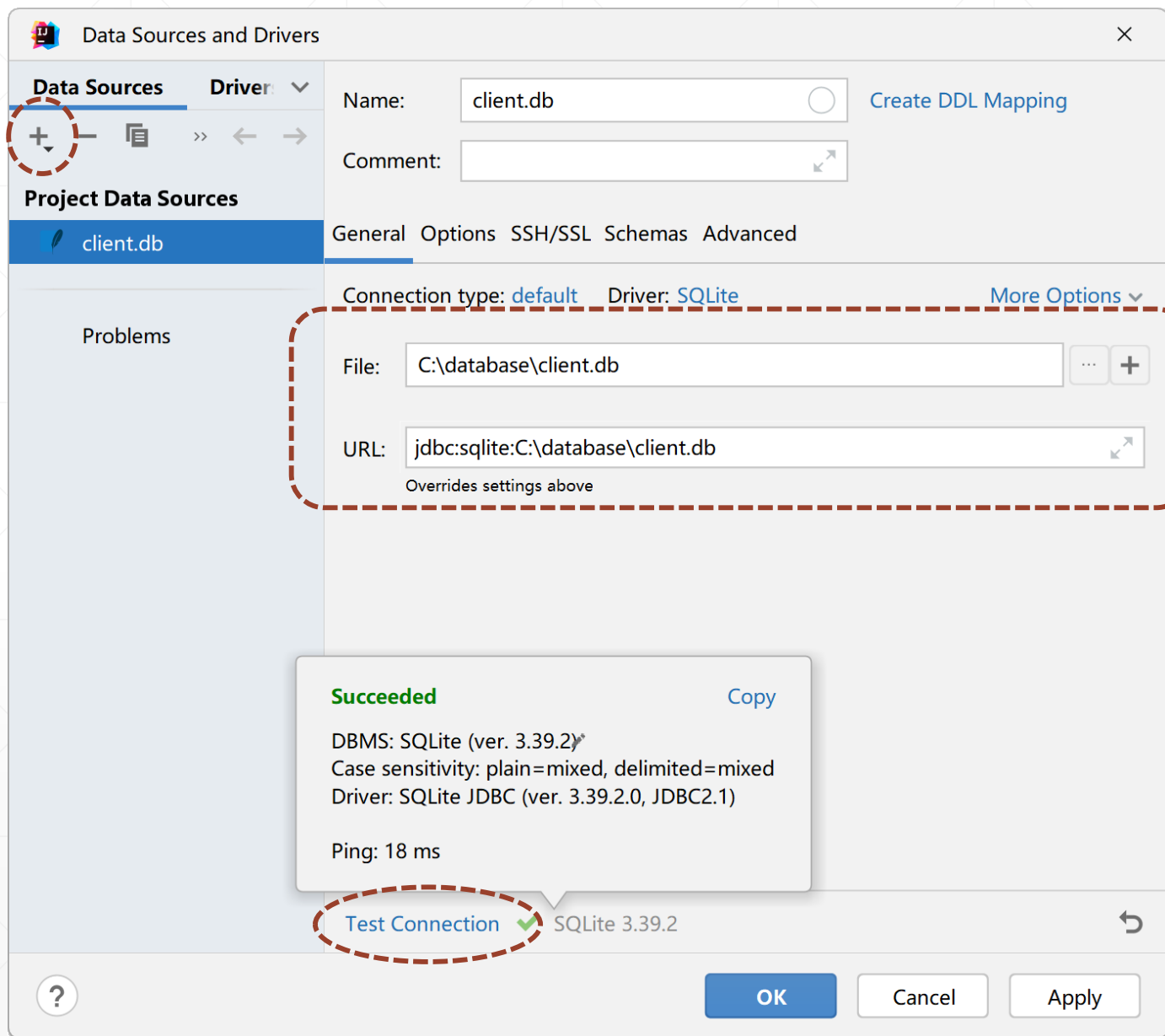
修改pom.xml，给JavaFX项目添加对于SQLite数据库的依赖：

```
<dependency>  
  <groupId>org.xerial</groupId>  
  <artifactId>sqlite-jdbc</artifactId>  
  <version>3.39.2.0</version>  
</dependency>
```

创建数据源

使用IntelliJ所提供的Database工具，创建数据源对象，以方便开发。

这一步并非必需的，只不过这一工具省去了单独安装数据库开发工具的麻烦，可以提升开发效率。



创建数据表

创建一个包含两个字段的数据表.....

Create New Table

×

Table:

OrderClient

Columns (3)

Keys (2)

Indexes

Foreign Keys

+

−

▲

▼

id

integer -- part of primary key

Name

text

Name:

Type:

Default:

Address

text

☐ Not null

☐ Auto inc

☐ Unique

☐ Primary key

▼ SQL Script

⚙

create table OrderClient

(

id integer not null

primary key autoincrement

unique,

Action:

Execute in database

?

Execute

Cancel

创建测试用记录

OrderClient			
4 rows			
WHERE		ORDER BY	
	id	Name	Address
1	1	张三	北京
2	2	李四	上海
3	3	王五	广州
4	4	赵六	深圳

Database	
client.db	1
main	
tables	3
OrderClient	
columns	3
id	INTEGER (auto increment)
Name	TEXT
Address	TEXT
keys	2
indexes	1
sqlite_master	
sqlite_sequence	
Server Objects	

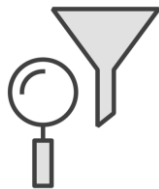
向表中添加几条测试用记录，下面就开始在项目中编写Java代码访问数据库.....

创建数据实体类

OrderClient		
m	nameProperty()	StringProperty
m	clone()	Object
m	addressProperty()	StringProperty
m	toString()	String
p	name	String
p	id	int
p	address	String

在项目中，针对要访问数据库表中的字段，定义OrderClient类，为了利用数据绑定特性，类中的属性，定义为JavaFX Bean Properties。

编写Repository, 实现CRUD



注意一下数据库的位置

```
public class OrderClientRepository {  
    //这里将数据库放到C:\database文件夹下  
    3 usages  
    private static String connectionStr = "jdbc:sqlite:C:\\database\\client.db";  
    //提取所有记录  
    2 usages  
    public List<OrderClient> getAllClients() {...}  
    //插入或更新记录  
    1 usage  
    public void insertOrUpdateClients(OrderClient client) {...}  
    //删除记录  
    1 usage  
    public void deleteClient(int clientId) {...}  
}
```

提取所有记录

```
public List<OrderClient> getAllClients() {  
    var clients = new ArrayList<OrderClient>();  
    String sql = "select * from OrderClient";  
    try (var conn :Connection = DriverManager.getConnection(connectionStr);  
         var statement :Statement = conn.createStatement();  
         var rs :ResultSet = statement.executeQuery(sql);  
    ) {  
        OrderClient client = null;  
        while (rs.next()) {  
            var id :int = rs.getInt( columnLabel: "id");  
            var name :String = rs.getString( columnLabel: "name");  
            var address :String = rs.getString( columnLabel: "address");  
            client = new OrderClient(id, name, address);  
            clients.add(client);  
        }  
    } catch (SQLException e) {  
        throw new RuntimeException(e);  
    }  
    return clients;  
}
```

插入或更新记录

当传入对象的id属性值为0时，是新增（因为此字段设置为数据库自增字段）。

```
public void insertOrUpdateClients(OrderClient client) {
    String sql = "";
    boolean isAddNew = true;
    if (client.getId() > 0) {
        //修改
        isAddNew = false;
        sql = String.format("update OrderClient set name='%s',address='%s' where id=%d",
            client.getName(), client.getAddress(), client.getId());
    } else {
        //新增
        sql = String.format("insert into OrderClient(name,address) values ('%s','%s')",
            client.getName(), client.getAddress());
    }
    try (var conn : Connection = DriverManager.getConnection(connectionStr);
        var statement : Statement = conn.createStatement();
    ) {
        statement.execute(sql);
        if (isAddNew) {
            sql = "select last_insert_rowid() as newId from OrderClient";
            var rs : ResultSet = statement.executeQuery(sql);
            if (rs.next()) {
                var newId : int = rs.getInt( columnLabel: "newId");
                client.setId(newId);
            }
        }
    } catch (SQLException e) {
        throw new RuntimeException(e);
    }
}
```

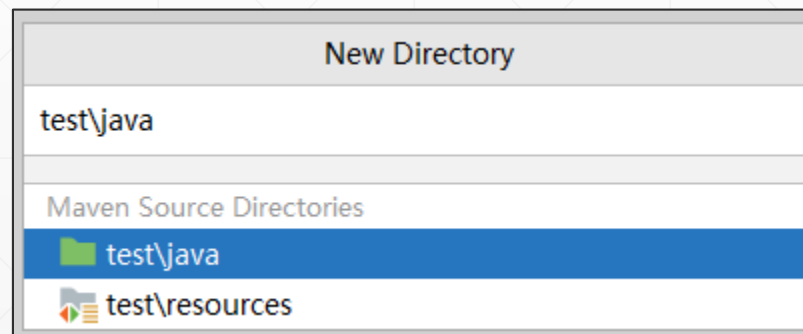
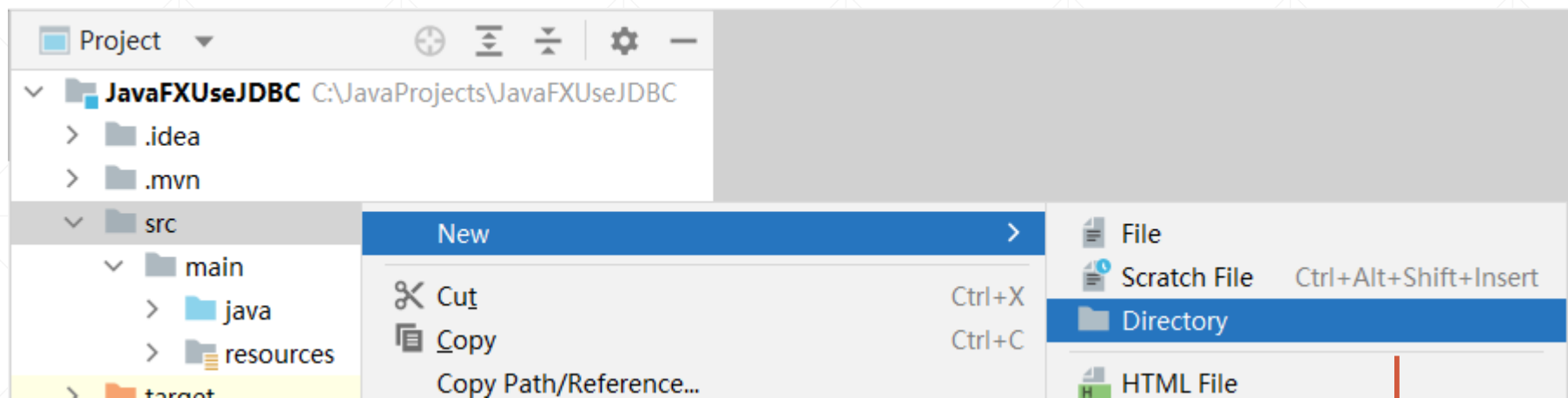
删除记录



```
1 usage
public void deleteClient(int clientId) {
    String sql = String.format("delete from OrderClient where id=%d", clientId);
    try (var conn :Connection = DriverManager.getConnection(connectionStr);
        var statement :Statement = conn.createStatement();
    ) {
        statement.execute(sql);
    } catch (SQLException e) {
        throw new RuntimeException(e);
    }
}
```

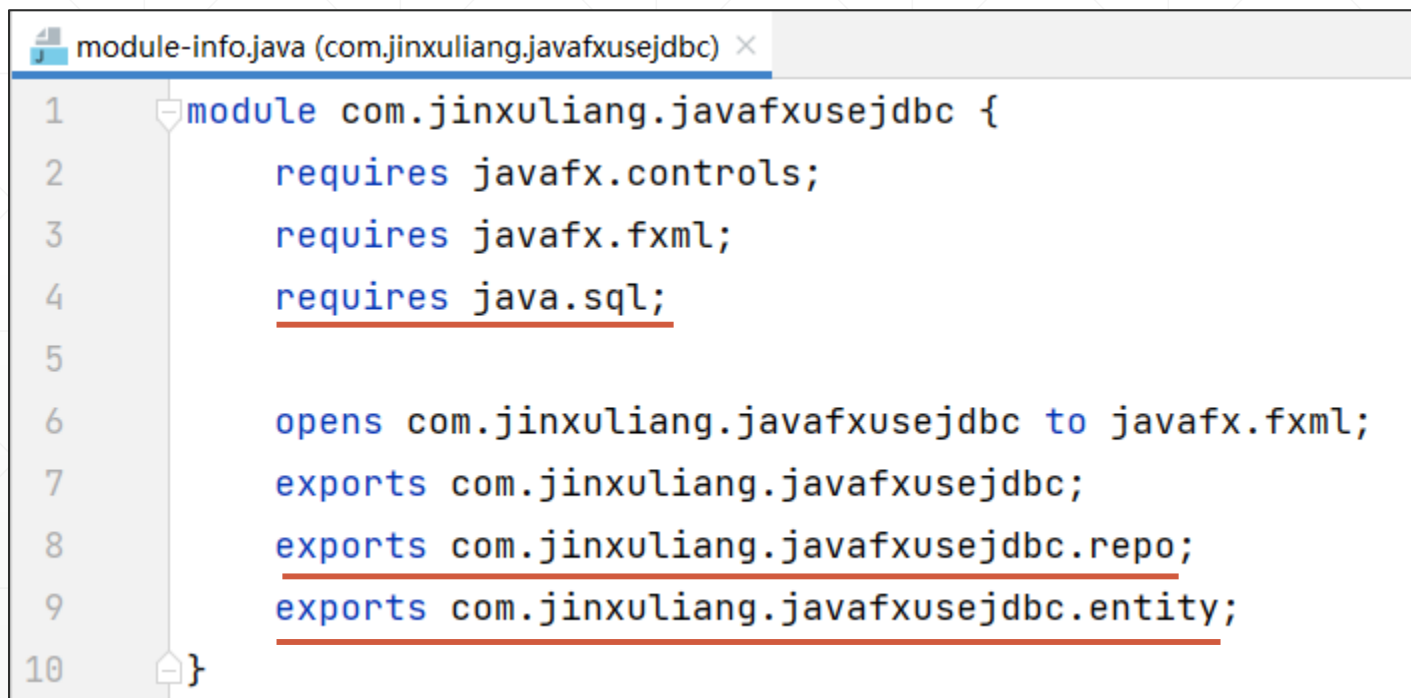
Repository编写完毕之后，通常我们会为其编写单元测试代码，以便检测其功能实现得是否正确.....

添加单元测试文件夹



一些容易疏忽的地方

由于JavaFX已经全部模块化了，因此，需要给module-info.java添加相应的声明



```
module-info.java (com.jinxuliang.javafxusejdbc) x
1  module com.jinxuliang.javafxusejdbc {
2      requires javafx.controls;
3      requires javafx.fxml;
4      requires java.sql;
5
6      opens com.jinxuliang.javafxusejdbc to javafx.fxml;
7      exports com.jinxuliang.javafxusejdbc;
8      exports com.jinxuliang.javafxusejdbc.repo;
9      exports com.jinxuliang.javafxusejdbc.entity;
10 }
```

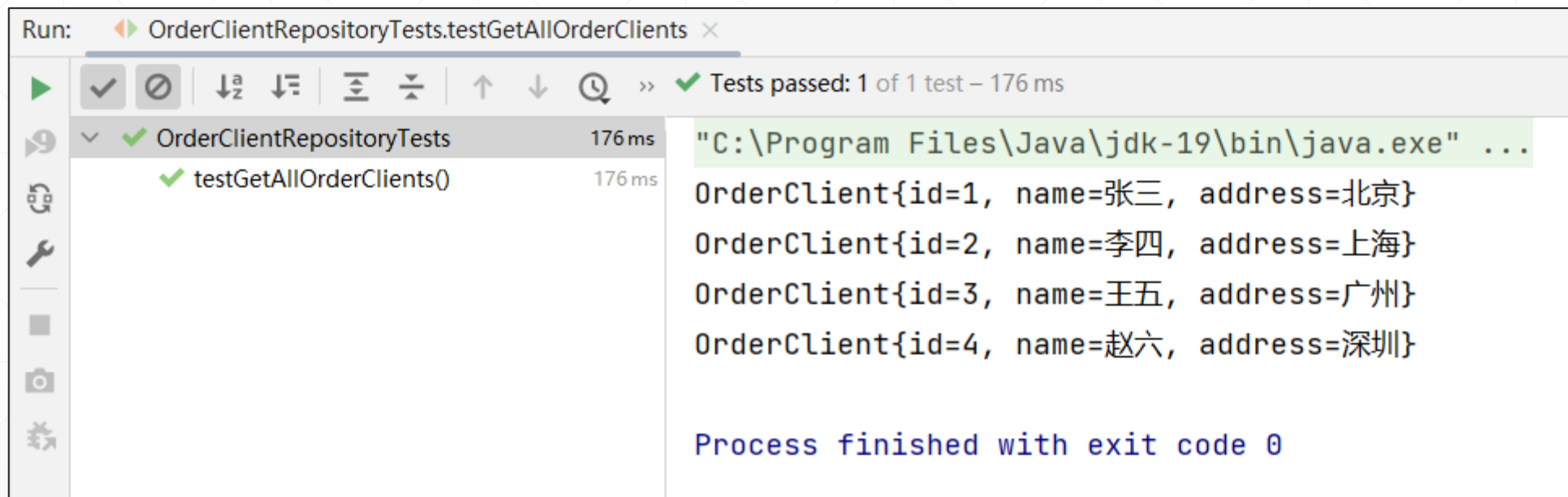
编写单元测试



```
public class OrderClientRepositoryTests {  
    2 usages  
    private static OrderClientRepository repository = null;  
  
    @BeforeAll  
    public static void init() {  
        repository = new OrderClientRepository();  
    }  
  
    @Test  
    public void testGetAllOrderClients() {  
        var clients : List<OrderClient> = repository.getAllClients();  
        Assertions.assertTrue( condition: clients.size() > 0);  
        clients.forEach(System.out::println);  
    }  
}
```

有关JUnit单元测试中各种注解的含义及用法，请参看本网站针对JUnit所开设的单元测试专题课程。

运行单元测试



下面开始设计UI层.....



id	name	address
1	张三	北京
2	李四	上海
3	王五	广州
4	赵六	深圳
13	钱七	重庆

修改记录

姓名

赵六

地址

深圳

取消

确定

新增

删除

修改

主要就是两个窗体，主窗体显示记录列表，另有一个对话框窗体实现新增或修改功能。

主窗体控制器-1

```
public class MainController implements Initializable {
```



FXML控件声明

//绑定数据源

6 usages

```
private ObservableList<OrderClient> clients = null;
```

//实现数据的CRUD

4 usages

```
private OrderClientRepository repository = null;
```

//用于引用新增或修改窗体

9 usages

```
private Stage newStage = null;
```

主窗体控制器-2

```
@Override
public void initialize(URL url, ResourceBundle resourceBundle) {
    //提取所有数据, 放到表格中显示
    repository = new OrderClientRepository();
    clients = FXCollections.observableList(repository.getAllClients());
    id.setCellValueFactory(new PropertyValueFactory<>(s: "id"));
    name.setCellValueFactory(new PropertyValueFactory<>(s: "name"));
    address.setCellValueFactory(new PropertyValueFactory<>(s: "address"));
    //设置TableView的数据源
    tableClient.setItems(clients);

    btnAdd.setOnAction(e -> {
        addOrModifyClient( isAddNew: true);
    });
    btnModify.setOnAction(e -> {
        addOrModifyClient( isAddNew: false);
    });
    btnDelete.setOnAction(e -> {
        var selectedIndex :int = tableClient.getSelectionModel().getSelectedIndex();
        var selectedOrderClient :OrderClient = clients.get(selectedIndex);
        repository.deleteClient(selectedOrderClient.getId());
        clients.remove(selectedIndex);
    });
}
```

配置TableView显示数据表中的所有记录

挂接事件响应代码

新增或修改-1



```
private void addOrModifyClient(boolean isAddNew) {  
    //加载布局文件  
    var fxmlLoader = new FXMLLoader(  
        JavaFXDbApplication.class.getResource("client-edit.fxml"));  
    try {  
        GridPane root = fxmlLoader.load();  
        //装载“新增 (或修改) 窗体”控制器  
        var controller = (EditClientController) fxmlLoader.getController();  
        //将主控制器引用传给“新增 (或修改) 窗体”控制器  
        controller.setMainController(this);  
        if (isAddNew) {  
            var newClient = new OrderClient(); //新增, 创建一个空白数据对象  
            controller.setOrderClient(newClient);  
        } else {  
            //修改, 取出当前选中的行所对应的对象  
            OrderClient client =  
                (OrderClient) tableClient.getSelectionModel().getSelectedItem();  
            if (client == null) return;  
            //克隆一份传给对话框窗体控制器  
            controller.setOrderClient((OrderClient) client.clone());  
        }  
    }  
}
```

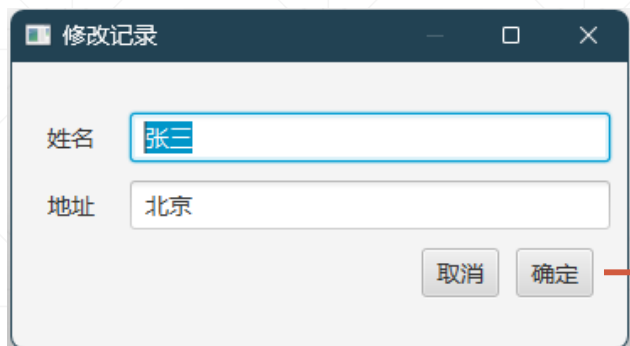
新增或修改-2



```
//显示对话框窗体
var scene = new Scene(root);
newStage = new Stage();
if (isAddNew) {
    newStage.setTitle("新增记录");
} else {
    newStage.setTitle("修改记录");
}
newStage.setScene(scene);
newStage.initModality(Modality.APPLICATION_MODAL);
newStage.showAndWait();
} catch (IOException ex) {
    throw new RuntimeException(ex);
} catch (CloneNotSupportedException e) {
    throw new RuntimeException(e);
}
}
```

只有关闭对话框窗体，
才能激活主窗体

对话框窗体的Ok按钮将回调主控制器的方法



```
//主窗体控制器提供将被对话框体口“回调”的方法
1 usage
public void onAddOrEditClient(boolean isAddNew, OrderClient client) {
    repository.insertOrUpdateClients(client);
    if (isAddNew) {
        //将新对象加入到Clients集合中
        clients.add(client);
    } else {
        OrderClient currentClient =
            (OrderClient) tableClient.getSelectionModel().getSelectedItem();
        //更新当前选中行所对应的数据对象
        currentClient.setName(client.getName());
        currentClient.setAddress(client.getAddress());
    }
    if (newStage != null) {
        newStage.close();
    }
}
```

主窗体控制器中的代码

对话框控制器代码解析



```
public class EditClientController implements Initializable {
```

```
//被新增或修改的对象
```

```
6 usages
```

```
private OrderClient orderClient = null;
```

```
//引用主控制器，以便回调它的方法
```

```
4 usages
```

```
private MainController mainController;
```

```
FXML控件声明
```

```
getter and setter
```

```
@Override
```

```
public void initialize(URL url, ResourceBundle resourceBundle) {
```

```
    btnCancel.setOnAction(e -> {  
        mainController.onCancelEdit();
```

```
    });
```

```
    btnOk.setOnAction(e -> {
```

```
        mainController.onAddOrEditClient(  
            isAddNew: orderClient.getId()==0, orderClient);
```

```
    });
```

```
}
```

```
//初始化数据绑定
```

```
1 usage
```

```
private void initDataBind() {
```

```
    if (orderClient != null) {
```

```
        txtName.textProperty().bindBidirectional(orderClient.nameProperty());
```

```
        txtAddress.textProperty().bindBidirectional(orderClient.addressProperty());
```

```
    }
```

```
}
```

建立双向绑定

回调主窗体的相应方法

小结



本讲通过一个典型示例，展示了以下技术要点：

- (1) JavaFX应用如何集成JDBC
- (2) 使用Repository封装CRUD，并为之编写单元测试
- (3) 基于JavaFX Bean Property和ObservableList<T>构建数据绑定源
- (4) 基于回调编程技巧，重用窗体实现“新增”和“修改”功能。



掌握了上述编程技巧，就可以举一反三地用于实际开发中。